Iniciando com oracledb

Oct 27, 2026 · @Pat

Para que serve

O `oracledb` é o driver oficial para o Python conversar com o banco Oracle. No modo padrão (Thin) **não precisa instalar o Oracle Client**.

```
pip install oracledb
```

O ciclo de sempre

```
import oracledb

conn = oracledb.connect(user="rm12345", password="...", dsn="host:1521/servico")  # 1) abre
cursor = conn.cursor()                                                            # 2) cria o cursor
cursor.execute("SELECT sysdate FROM dual")                                        # 3) manda o SQL
resultado = cursor.fetchone()                                                     # 4) lê o resultado
cursor.close()                                                                    # 5) fecha
conn.close()
```

Peça	O que é
conexão (conn)	O "telefone ligado" com o banco
cursor	Quem executa os comandos SQL e guarda os resultados
DSN	O endereço do banco: `servidor:porta/serviço`

Sempre feche o cursor e a conexão no fim. Use `try ... finally` para fechar mesmo quando dá erro.

Ler dados (SELECT)

O `execute` manda o SQL, e o `fetch` busca o resultado.

```
cursor.execute("SELECT quote_text, author_name FROM quotes")

cursor.fetchone()    # UMA linha (tupla), ou None se acabou
cursor.fetchall()    # TODAS as linhas (lista de tuplas)
```

```
linhas = cursor.fetchall()
# [("Frase A", "Ana"), ("Frase B", "Bruno")]
for citacao, autor in linhas:
    print(autor, citacao)
```

- Cada linha vem como **tupla**, na ordem das colunas do SELECT.
- Se não houver linhas, `fetchall()` devolve lista vazia `[]` e `fetchone()` devolve `None`.
- O `fetchone()` consome a linha: chamar de novo traz a próxima.

Passar valores sem erro: bind variables

Nunca cole valores direto no SQL com f-string: isso abre a porta para SQL injection e quebra com aspas no texto (como em `It's`). Use os "buracos" `:1`, `:2` e passe os valores separados.

```
# posicional: os valores vão numa tupla, na ordem
cursor.execute("SELECT * FROM quotes WHERE author_name = :1", ("Ana",))
```

```
# nomeado: os valores vão num dicionário
cursor.execute("SELECT * FROM quotes WHERE author_name = :autor", {"autor": "Ana"})
```

Estilo	SQL	Valores
Posicional	`:1`, `:2`	tupla, na ordem dos buracos
Nomeado	`:autor`, `:texto`	dicionário, a chave casa com o nome

Com um valor só, a tupla precisa da vírgula: `("Ana",)`. Sem ela, os parênteses não criam tupla.

Buscar por parte do texto

Para achar um trecho (como na pesquisa do projeto), use `LIKE` com `%` e coloque os `%` no **valor**, e não no SQL:

```
cursor.execute("SELECT quote_text FROM quotes WHERE UPPER(author_name) LIKE UPPER(:1)",
               (f"%{texto}%",))
```

O `UPPER` nos dois lados faz a busca ignorar maiúsculas e minúsculas.

Gravar dados (INSERT, UPDATE, DELETE)

Comandos que alteram o banco só ficam salvos depois do `commit`.

```
cursor.execute("INSERT INTO quotes (quote_text, author_name) VALUES (:1, :2)",
               ("Frase A", "Ana"))
conn.commit()          # confirma. SEM commit, nada fica salvo
```

Várias linhas de uma vez, com `executemany` (roda o mesmo SQL uma vez para cada tupla da lista):

```
dados = [("Frase A", "Ana"), ("Frase B", "Bruno")]
cursor.executemany("INSERT INTO quotes (quote_text, author_name) VALUES (:1, :2)", dados)
conn.commit()
```

O `executemany` é mais rápido do que um `for` com `execute`, porque envia tudo ao banco numa única ida.

Comando	O que faz
`conn.commit()`	**Confirma** as alterações no banco
`conn.rollback()`	**Desfaz** o que ainda não foi confirmado
`cursor.rowcount`	Quantas linhas o último comando afetou

Padrão seguro

```
try:
    cursor.executemany(sql, dados)
    conn.commit()                    # deu tudo certo: confirma
except Exception as e:
    conn.rollback()                  # algo falhou: desfaz
    print(f"Erro ao inserir: {e}")
finally:
    cursor.close()                   # sempre fecha, com erro ou sem
    conn.close()
```

Tratar erros

```
try:
    conn = oracledb.connect(user=user, password=senha, dsn=dsn)
except oracledb.DatabaseError as e:
    erro, = e.args                # tira o objeto de erro da tupla
    print(erro.message)           # texto do erro
    print(erro.code)              # número do ORA (ex.: 1017)
```

O que é o `e.args`

Toda exceção do Python guarda o que recebeu na criação numa **tupla** chamada `args`. No `oracledb`, essa tupla tem **um único item**: um objeto de erro (da classe `oracledb.errors._Error`) com os detalhes.

```
erro, = e.args      # desempacota: pega o item de dentro da tupla
erro.message        # texto completo do erro
erro.full_code      # "DPY-6005" ou "ORA-01017"
erro.code           # número do ORA (1017, 955...); nos erros DPY vem 0
```

A vírgula em `erro, = e.args` diz "a tupla tem um item só, coloque-o em `erro`". Sem a vírgula, `erro` seria a tupla inteira, e `erro.message` daria erro. Se você só quer mostrar o texto, `print(e)` já resolve; o `erro, = e.args` é necessário quando você quer o **número** do erro (`erro.code`).

O objeto de erro tem outros campos além desses (como `offset` e `isrecoverable`). A lista pode variar entre versões do `oracledb`, por isso o uso mais seguro é `message` e `code`. Para ver todos na sua versão, rode `print(vars(erro))`.

Dois tipos de erro

- **`DPY-xxxx`:** nasceu no driver Python, antes de chegar ao banco (rede, DSN errado, porta fechada). O `erro.code` vem `0`.
- **`ORA-xxxxx`:** veio do banco Oracle. O `erro.code` traz o número, como `1017`.

Erros que você vai ver

Erro	Significa
`ORA-01017`	Usuário ou senha inválidos
`ORA-00942`	Tabela ou view não existe
`ORA-00955`	Já existe um objeto com esse nome
`DPY-6005`	Não conseguiu conectar (host/porta errados, banco fora do ar)

Em alguns casos (como um host que não existe), o erro pode vir como erro de rede do Python (`OSError`), e não como `DatabaseError`. Por isso vale tratar os dois.

Particularidades do Oracle e segurança

Diferenças em relação a outros bancos

- **`SELECT` sempre precisa de `FROM`:** sem tabela, use `FROM dual` (ex.: `SELECT sysdate FROM dual`).
- **Não existe `CREATE TABLE IF NOT EXISTS`** nas versões mais comuns: tente criar e trate o erro `ORA-00955` ("já existe").
- **`TRUNCATE TABLE`** apaga tudo de uma vez e **não dá para desfazer**. Já o `DELETE` pode ser desfeito com `rollback` antes do `commit`.
- **`id` automático:** `NUMBER GENERATED ALWAYS AS IDENTITY`.
- **Busca sem diferenciar maiúscula:** `WHERE UPPER(coluna) LIKE UPPER(:1)`, com `%texto%`.

```
CREATE TABLE quotes (
    id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    quote_text  VARCHAR2(1000) NOT NULL,
    author_name VARCHAR2(200)  NOT NULL
)
```

Segurança

- **Nunca coloque a senha no código.** Leia de um arquivo de configuração (como o `config.json`) ou peça no terminal com `getpass`, e deixe o arquivo no `.gitignore`.
- **Use sempre bind variables** nos valores, e nunca f-string dentro do SQL.
- **Sempre feche a conexão** (`try ... finally`), para não deixar sessões abertas no banco.
- Use um usuário do banco com **permissão mínima** para o que o programa precisa.